FetalAI: Predictive Fetal Health Monitoring

Name : Sanket Manoj Kokate
Roll : 190 PRN : 2022011031093
B.Tech CSE
D.Y.P.-A.T.U., Talsande.

1. Project Overview

FetalAI is a machine learning-based web application designed to predict and monitor fetal health during pregnancy. By analyzing 22 distinct parameters from Cardiotocogram (CTG) exams (such as fetal heart rate, accelerations, and decelerations), the system classifies fetal health into three categories: **Normal**, **Suspect**, or **Pathological**. This provides healthcare professionals with an objective tool for early intervention and remote monitoring.

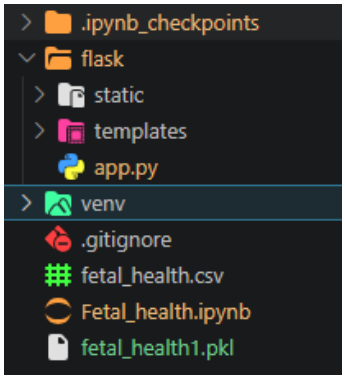
2. Technical Architecture

- **Language:** Python 3.x
- **Data Processing & Analysis:** Pandas, NumPy, Matplotlib, Seaborn
- **Machine Learning:** Scikit-learn (Random Forest, Decision Tree, Logistic Regression, KNN)
- **Data Balancing:** Imbalanced-learn (SMOTE)
- **Web Framework:** Flask
- **Frontend:** HTML5, Tailwind CSS

3. Step-by-Step Development Procedure

Phase 1: Environment Setup and Data Collection

1. **Directory Initialization:** Created the main project structure (FETAL_HEALTH) alongside a flask subdirectory for deployment.



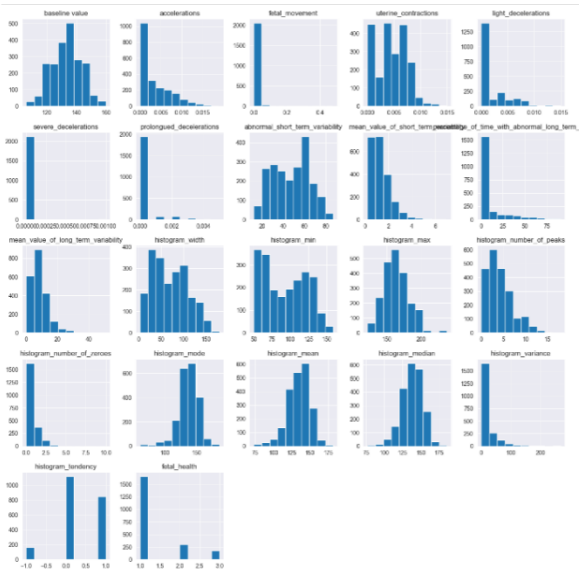
2. **Virtual Environment:** Initialized a Python virtual environment to manage dependencies securely.
3. **Data Acquisition:** Downloaded the fetal_health.csv dataset (2,126 records) from Kaggle and loaded it into a Jupyter Notebook using Pandas.

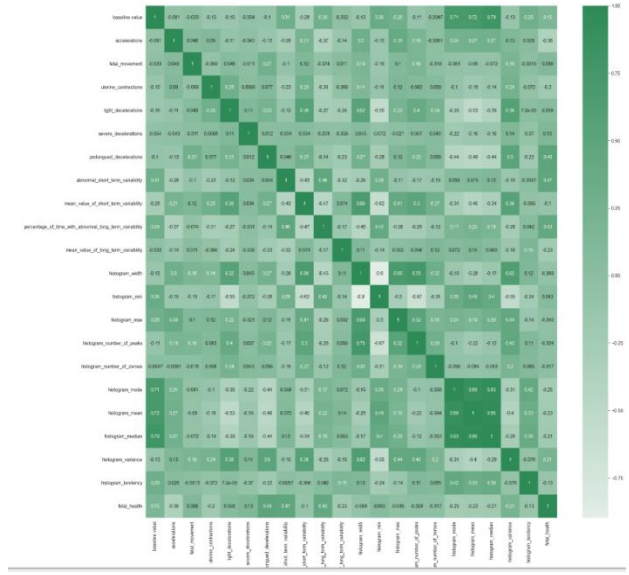
Phase 2: Exploratory Data Analysis (EDA)

1. **Descriptive Statistics:** Utilized data.describe() and data.info() to verify data integrity, confirming the absence of null values and categorical variables.

	count	mean	std	min	25%	50%	75%	max
baseline value	2126.0	133.303857	9.840844	106.0	126.000	133.000	140.000	160.000
accelerations	2126.0	0.003178	0.003866	0.0	0.000	0.002	0.006	0.019
fetal_movement	2126.0	0.009481	0.046666	0.0	0.000	0.000	0.003	0.481
uterine_contractions	2126.0	0.004366	0.002946	0.0	0.002	0.004	0.007	0.015
light_decelerations	2126.0	0.001889	0.002960	0.0	0.000	0.000	0.003	0.015
severe_decelerations	2126.0	0.000003	0.000057	0.0	0.000	0.000	0.000	0.001
prolongued_decelerations	2126.0	0.000159	0.000590	0.0	0.000	0.000	0.000	0.005
abnormal_short_term_variability	2126.0	46.990122	17.192814	12.0	32.000	49.000	61.000	87.000
mean_value_of_short_term_variability	2126.0	1.332785	0.883241	0.2	0.700	1.200	1.700	7.000
percentage_of_time_with_abnormal_long_term_variability	2126.0	9.846660	18.396880	0.0	0.000	0.000	11.000	91.000
mean_value_of_long_term_variability	2126.0	8.187629	5.628247	0.0	4.600	7.400	10.800	50.700
histogram_width	2126.0	70.445908	38.955693	3.0	37.000	67.500	100.000	180.000
histogram_min	2126.0	93.579492	29.560212	50.0	67.000	93.000	120.000	159.000
histogram_max	2126.0	164.025400	17.944183	122.0	152.000	162.000	174.000	238.000
histogram_number_of_peaks	2126.0	4.068203	2.949386	0.0	2.000	3.000	6.000	18.000
histogram_number_of_zeroes	2126.0	0.323612	0.706059	0.0	0.000	0.000	0.000	10.000
histogram_mode	2126.0	137.452023	16.381289	60.0	129.000	139.000	148.000	187.000
histogram_mean	2126.0	134.610536	15.593596	73.0	125.000	136.000	145.000	182.000
histogram_median	2126.0	138.090310	14.466589	77.0	129.000	139.000	148.000	186.000
histogram_variance	2126.0	18.808090	28.977636	0.0	2.000	7.000	24.000	269.000
histogram_tendency	2126.0	0.320320	0.610829	-1.0	0.000	0.000	1.000	1.000
fetal_health	2126.0	1.304327	0.614377	1.0	1.000	1.000	1.000	3.000

2. **Visual Analysis:** * Deployed histograms to understand the univariate distribution of features.
   - Utilized scatter plots for bivariate analysis.
   - Generated a Seaborn correlation heatmap to identify relationships between the 22 features and the target variable





3. **Class Imbalance Check:** Identified a significant skew toward the 'Normal' class (1.0) compared to 'Suspect' (2.0) and 'Pathological' (3.0).

Phase 3: Data Preprocessing and Feature Selection

1. **Feature Reduction:** Dropped the histogram_mean column based on correlation insights, isolating the top 8 most influential features for model training.
2. **Data Scaling:** Applied MinMaxScaler to normalize the data, ensuring features with larger numerical ranges did not disproportionately influence the algorithms.
3. **Data Balancing:** Applied the Synthetic Minority Over-sampling Technique (SMOTE) to the training split to synthesize minority class data, resulting in an equally balanced training set.

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=10)
print(X_train.shape, X_test.shape)
```

```
(1488, 20) (638, 20)
```

Apply SMOTE

```
from imblearn.over_sampling import SMOTE
smote = SMOTE()
X_train_smote, y_train_smote = smote.fit_resample(X_train.astype('float'), y_train)

from collections import Counter
print("Before SMOTE : ", Counter(y_train))
print("After SMOTE : ", Counter(y_train_smote))
```

```
Before SMOTE :  Counter({1.0: 1158, 2.0: 201, 3.0: 129})
After SMOTE :  Counter({1.0: 1158, 2.0: 1158, 3.0: 1158})
```

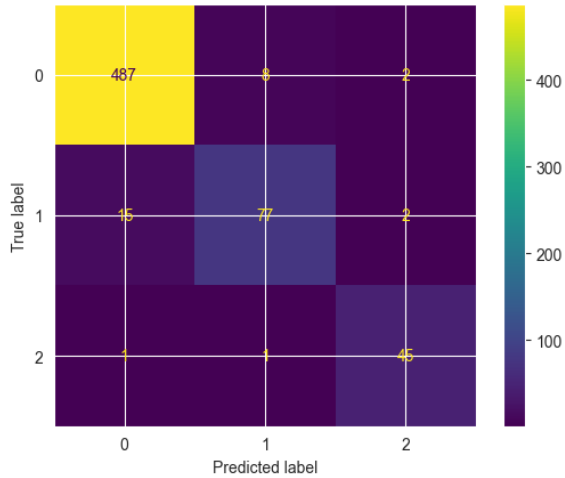
Training the model in multiple algorithms Now our data is cleaned and it's time to build the model. We can train our data on different algorithms. For this project we are applying three classification algorithms. The best model is saved based on its performance.

1 Random Forest

Phase 4: Model Building and Evaluation

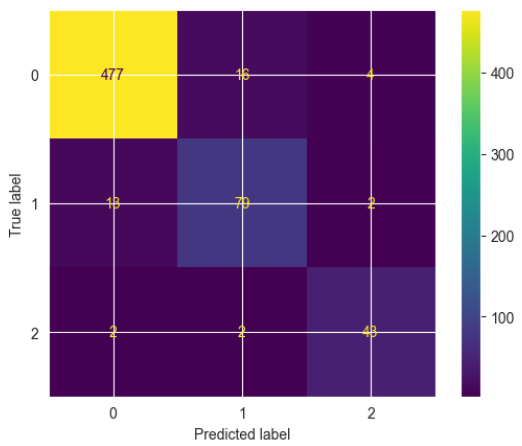
1. **Algorithm Training:** Trained four distinct classification algorithms on the SMOTE-balanced data:
   - Random Forest Classifier

```
0.9545454545454546
For the amounts of training data is:  3474
Accuracy of RandomForestClassifier:  0.9545454545454546
```



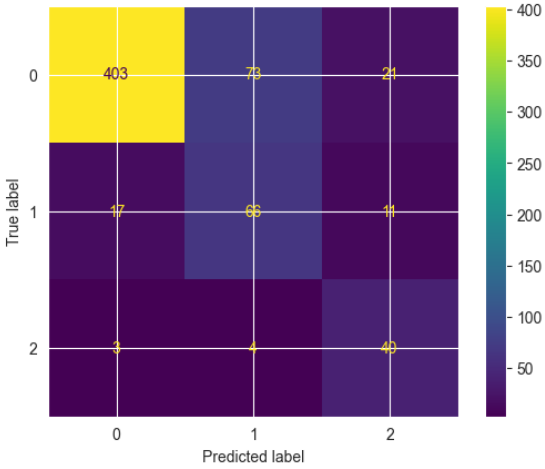
   - Decision Tree Classifier

```
0.9388714733542319
For the amounts of training data is:  3474
Accuracy of DecisionTreeClassifier:  0.9388714733542319
```



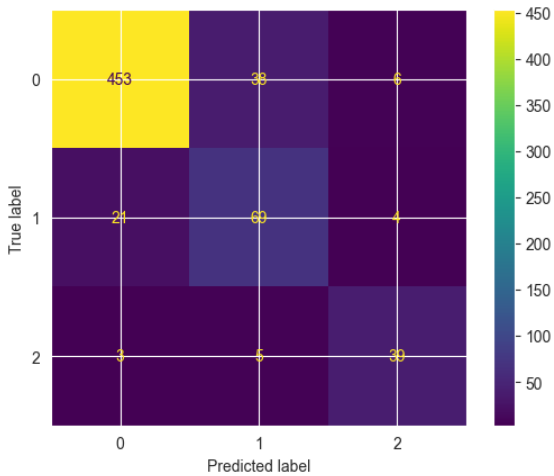
   - Logistic Regression

```
0.7978056426332288
For the amounts of training data is:  3474
Accuracy of LogisticRegression:  0.7978056426332288
```



- K-Nearest Neighbors (KNN)

```
0.8793103448275862
For the amounts of training data is:  3474
Accuracy of KNeighborsClassifier:  0.8793103448275862
```

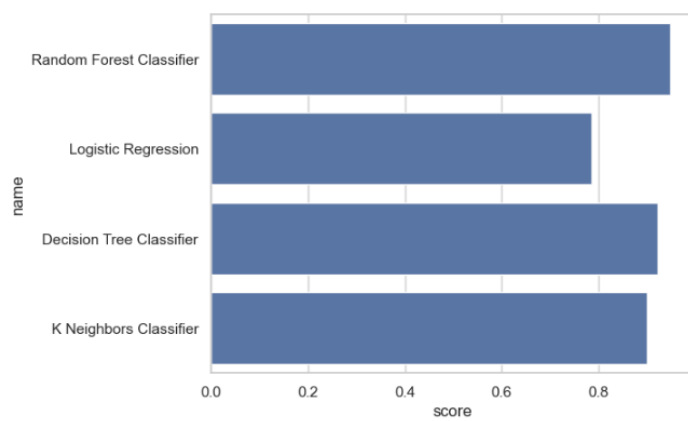


2. **Performance Testing:** Evaluated each model against the test set using accuracy scores, confusion matrices, and classification reports.

3. **Model Selection:** The **Random Forest Classifier** emerged as the highest-performing model, achieving approximately **95% accuracy**.

	name	score
0	Random Forest Classifier	0.948276
1	Logistic Regression	0.785266
2	Decision Tree Classifier	0.923197
3	K Neighbors Classifier	0.899687

```
sns.set(style="whitegrid")
ax = sns.barplot(y="name", x="score", data=df)
plt.show()
```



Phase 5: Model Deployment

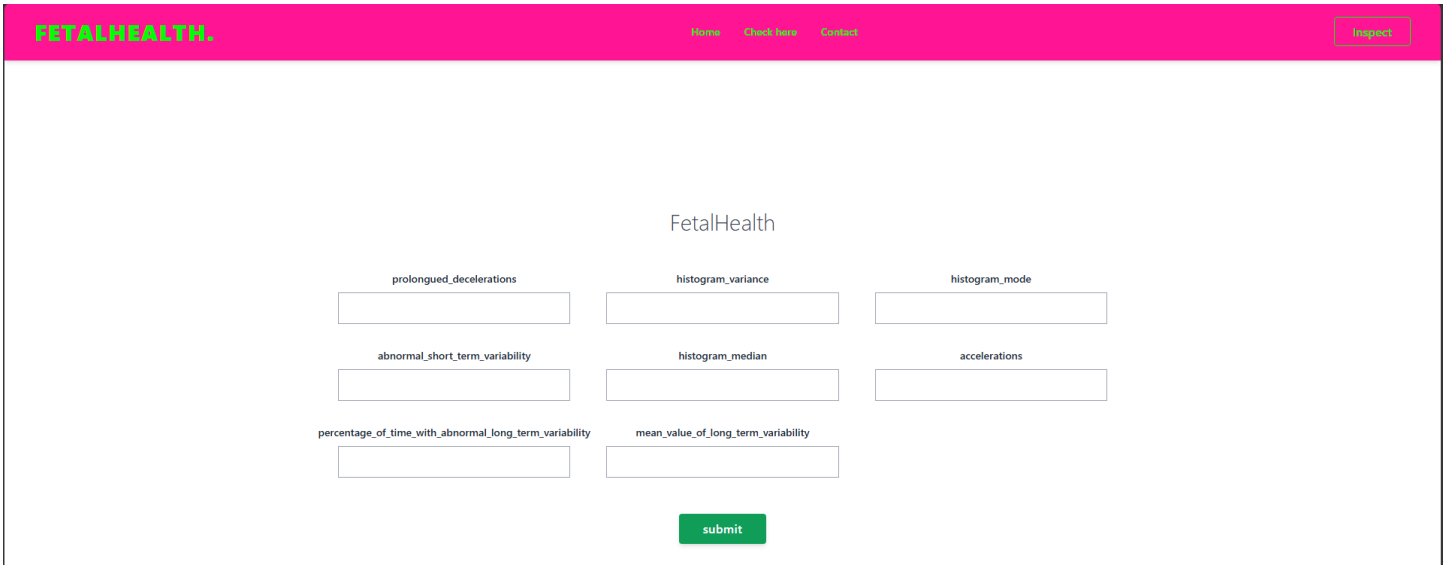
1. **Serialization:** Saved the trained Random Forest model using the pickle library, exporting it as fetal_health1.pkl to prevent the need for retraining on every server initialization.
2. **Web Server Setup:** Constructed app.py using Flask. Configured a POST route to receive user inputs from the web interface, process them into the expected 8-feature 2D array, and pass them to the loaded .pkl model for prediction.
3. **Result Mapping:** Mapped the numerical model outputs (1, 2, 3) back to their readable clinical labels ('Normal', 'Suspect', 'Pathological').

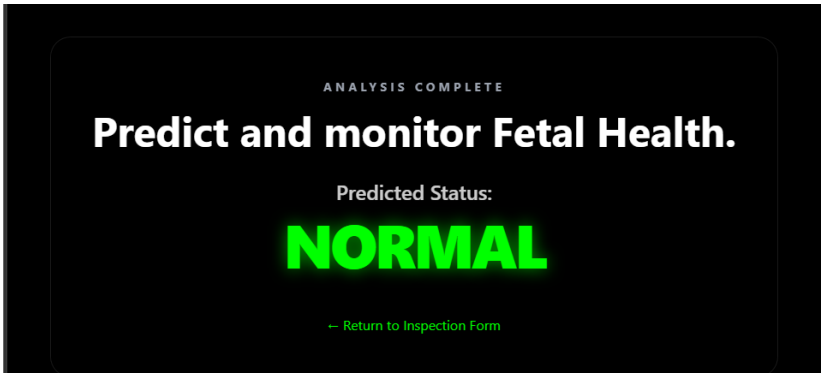
Phase 6: Frontend Interface Development

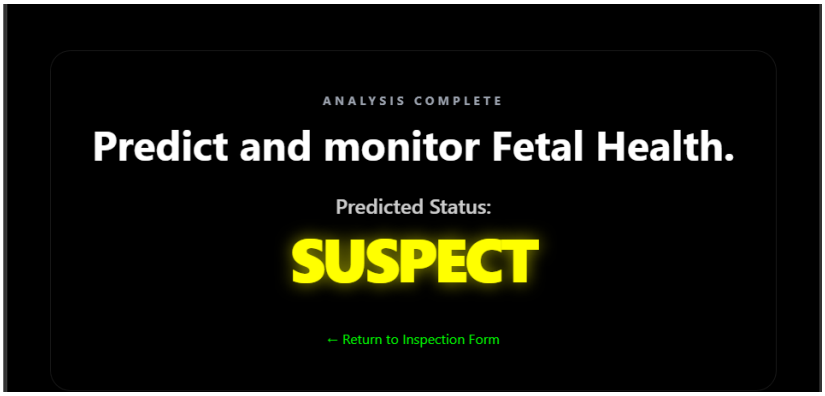
Developed a responsive, modern UI using HTML and Tailwind CSS via CDN.

1. **Index (index.html):** A high-impact landing page introducing the FetalAI tool.
2. **Inspect (inspect.html):** A clean, 3-column grid form allowing medical staff to input the 8 required CTG metrics.
3. **Output (output.html):** A dynamic results page that utilizes conditional styling (Neon Green, Yellow, Pink) to clearly display the predicted health status.
4. **Contact (contact.html):** A standard support page detailing clinical locations and support channels.

UI :







4. How to Run the Application

1. Navigate to the flask directory within the project root.
2. Ensure the virtual environment is activated and dependencies are installed.
3. Verify that fetal_health1.pkl and app.py are in the same directory, and the HTML files are located inside a templates folder.
4. Execute the application via the terminal: python app.py
5. Open a web browser and navigate to http://127.0.0.1:5000/ to access the FetalAI dashboard.